

SGD with Momentum Explained

Optimizers in Deep Learning — Part 2 | Notes based on the CampusX video

1. The Problem: Zig-Zag Path of Gradient Descent

When training a neural network using mini-batch (or stochastic) gradient descent, the path the optimizer takes towards the minimum is rarely a straight line. Instead, it oscillates back and forth, tracing a zig-zag pattern, especially when the loss surface is elongated (like a narrow valley).

This zig-zagging happens because gradients computed on small batches of data are noisy estimates of the true gradient. Each step nudges the parameters in a slightly different direction — some directions cancel out, while others reinforce each other.

The consequence: a lot of the "movement" is wasted oscillating sideways, and only a small part of it actually contributes to progress toward the minimum. This makes training slower than it needs to be.

2. The Idea Behind Momentum

Momentum Optimization smooths out this zig-zag path, making it straighter and more direct. It does this by using an **Exponentially Weighted Moving Average (EWMA)** of past gradients instead of relying only on the current gradient.

Intuitively, think of a ball rolling down a hill. Instead of reacting only to the slope right under it at each instant, the ball carries **momentum** from its previous motion. This momentum:

- Cancels out oscillations in directions that keep flipping back and forth (e.g. the vertical direction in a 2D loss surface).
- Reinforces and accelerates movement in directions that are consistent across steps (e.g. the horizontal direction toward the minimum).

Net effect: the zig-zag path becomes a much straighter, faster route to the minimum — reducing the overall time required to train the model.

3. Mathematical Formulation

Momentum introduces a new quantity, often called the **velocity** (v), which accumulates an exponentially weighted average of past gradients:

$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot \nabla L(w_t)$$

$$w_{t+1} = w_t - \alpha \cdot v_t$$

Where:

Symbol	Meaning
--------	---------

w_t	Model parameters (weights) at step t
$\nabla L(w_t)$	Gradient of the loss function at step t
v_t	Velocity (EWMA of gradients) at step t
β (beta)	Momentum coefficient, typically 0.9 (controls how much past gradients are remembered)
α (alpha)	Learning rate

Note: This is the EWMA form (with weight $1 - \beta$ on the new gradient). Some libraries (e.g. PyTorch's default SGD with momentum) use a slightly different but equivalent-in-spirit form without the $(1 - \beta)$ scaling. The underlying intuition is the same.

4. Effect of the Beta (β) Hyperparameter

β controls how much "memory" of past gradients is carried forward, and it has a direct effect on training behavior:

β value	Effect
$\beta = 0$	No momentum at all — reduces to plain gradient descent.
$\beta \approx 0.5\text{--}0.8$	Mild smoothing; some reduction in zig-zag, faster than vanilla GD.
$\beta \approx 0.9$ (common default)	Strong smoothing; significantly straighter path and faster convergence.
$\beta \rightarrow 1$	Too much memory — the optimizer can overshoot the minimum and oscillate around it, or even diverge.

In practice, $\beta = 0.9$ is the most commonly used default, as it strikes a good balance between smoothing out noise and staying responsive to the true direction of descent.

5. Convex vs Non-Convex Cost Functions

Understanding momentum's behavior is closely tied to the shape of the cost (loss) function being optimized:

- **Convex function:** Has a single global minimum, and its shape looks like a smooth bowl. Any local minimum is also the global minimum, so gradient-based methods reliably converge to the best solution.
- **Non-convex function:** Has multiple local minima, saddle points, and plateaus. Neural network loss surfaces are typically non-convex, which makes optimization harder — an optimizer can get stuck in a local minimum or slow down drastically on a plateau.

Momentum is particularly useful on non-convex surfaces: the accumulated velocity can help the optimizer push through small bumps, saddle points, and plateaus that would otherwise stall plain gradient descent.

6. Problems with Momentum Optimization

- **Overshooting:** Because velocity accumulates, the optimizer can carry too much speed and overshoot past the minimum, requiring extra steps to correct course.
- **Oscillation near the minimum:** Near convergence, momentum can cause the parameters to oscillate around the minimum rather than settling smoothly into it.
- **Extra hyperparameter:** β needs to be tuned; a poor choice can hurt convergence instead of helping it.
- **Not adaptive:** Plain momentum applies the same logic uniformly, unlike more advanced optimizers (e.g. Adam, RMSProp) that adapt the learning rate per parameter.

7. Summary

Momentum optimization speeds up training by smoothing the zig-zag path of mini-batch gradient descent. It does this by maintaining an exponentially weighted moving average of past gradients (velocity), which cancels out oscillations in inconsistent directions while reinforcing movement in the consistent direction toward the minimum. The momentum coefficient β (commonly 0.9) controls the strength of this smoothing effect, and while momentum generally accelerates convergence, it can also introduce overshooting if not tuned carefully.

Reference: Video Timestamps

Source video: “SGD with Momentum Explained in Detail with Animations | Optimizers in Deep Learning Part 2” (CampusX)

00:00 — Intro

01:24 — Understanding Graphs

07:13 — Image Representation

- 10:41** — Convex vs Non-Convex Optimization
- 18:13** — Momentum Optimization
- 20:49** — The What?
- 24:23** — How to Implement the Concept Mathematically
- 28:41** — Effect of Beta
- 34:33** — Problems with Momentum Optimization
- 35:54** — Visualization
- 38:04** — Outro

Notes compiled for personal study based on the CampusX YouTube lecture on Momentum Optimizer in Deep Learning. For the full explanation with animations, refer to the original video.